Aptos On-Chain Wordle

Technical Summary

Wordle Web3 Project

October 2026

1 System Overview

The project implements Wordle with an Aptos Move contract and a React frontend. Players receive six attempts to guess a five-letter word. Petra mode signs live transactions with an Aptos wallet, burner mode uses a browser-managed development account, and simulation mode runs entirely locally.

The contract is authoritative in live modes. It stores the current answer, attempt history, game outcome, and cumulative statistics. The frontend provides validation, transaction construction, board rendering, and wallet management. Frontend validation improves the user experience; it does not replace contract security.

The principal files are:
`contracts/sources/wordle.move`, the game contract; `frontend/src/App.jsx`, the game lifecycle; `frontend/src/utils/vali`
the frontend dictionary; and `contracts/deploy.sh`, deployment automation.

2 Move Contract

The module uses the named address `wordle_addr`. Its configuration is stored under the module address:

```
struct WordleConfig has key {
    admin: address,
    word_list: vector<vector<u8>>,
    is_active: bool,
}
```

`initialize` creates this resource once and installs a five-letter answer pool. A second initialization is rejected by the existing-resource check. The global answer pool allows multiple players to receive different randomized targets.

Each player has an account-owned `PlayerState`:

```
struct PlayerState has key {
    attempts: vector<GuessResult>, is_solved: bool,
    max_attempts: u64, target_word: vector<u8>,
    games_played: u64, games_won: u64,
    current_streak: u64, best_streak: u64,
    guess_distribution: vector<u64>, game_finished: bool,
}
```

A GuessResult contains the submitted byte vector and its feedback vector. Feedback uses `0` for absent, `1` for misplaced, and `2` for exact.

3 Game Lifecycle

Start. `start_game` gets the configuration, selects an answer index with Aptos randomness, and creates or resets the player's state. Resetting clears attempts and current-game flags but preserves cumulative statistics.

If W is the answer pool and R is the bounded random index:

$$\texttt{target_word} = W[R], \qquad 0 \le R < |W|.$$

Submit. `submit_guess` verifies that the player has a game, the guess contains exactly five bytes, the game is active, and fewer than six attempts have been used. It evaluates the guess, appends the result, and checks the outcome.

A game wins when all five feedback values are exact:

$$\forall i \in \{0, 1, 2, 3, 4\} : \ R_i = 2.$$

A win increments games played, games won, and the streak. It also increments the distribution index corresponding to the winning attempt:

$$d_{n-1} \leftarrow d_{n-1} + 1,$$

where n is the attempt number. The best streak is updated as

$$\texttt{best_streak} = \max(\texttt{current_streak}, \texttt{best_streak}).$$

A loss on the sixth unsuccessful guess increments games played and resets the current streak. The final loss does not add to the guess distribution. Game settlement uses `game_finished`, preventing one completed game from updating statistics twice. Starting another game resets only the current game.

4 Letter Evaluation

Feedback is calculated in two passes to handle duplicate letters correctly.

Pass one: exact matches. Every position i where $G_i = T_i$ becomes exact, and its target position is consumed. This prevents a duplicate letter elsewhere from claiming that position.

Pass two: misplaced matches. Each remaining guess letter searches the unconsumed target letters. A match becomes misplaced and is consumed once. Remaining letters are reported absent.

For example, if `APPLE` is the target and `POPPA` is the guess, the result is $[1, 0, 2, 0, 1]$. The exact middle `P` is assigned first; duplicate occurrences then receive feedback only if target occurrences remain.

5 Contract Views

The frontend reads contract state through Move views:

Function	Purpose
`has_game(address)`	Tests whether player state exists
`get_player_status(address)`	Returns solved state, attempt count, and limit
`get_attempt_at(address,index)`	Returns a submitted word and feedback
`get_target_word(address)`	Returns the answer for reveal
`get_player_stats(address)`	Returns statistics and guess distribution

After each transaction is committed, the frontend waits for Aptos finality and reloads these views. Game mode and wallet-address guards prevent stale asynchronous responses from changing the wrong session.

Byte vectors are converted to strings for the board and feedback vectors are decoded into values from zero to two. The frontend derives whether the live game is active, solved, or exhausted from the loaded attempts and contract status.

6 Frontend Options and Modes

The header provides an operating-mode selector for Petra, burner, and simulation modes. Help, statistics, wallet, reset, and start-game controls are available separately.

Before submission, the frontend converts the guess to uppercase and verifies that it has exactly five characters, belongs to `VALID_WORDS`, has not already been submitted, and belongs to an active game.

6.1 Petra Mode

Petra mode is the standard option for live gameplay. Selecting it presents the wallet connection controls. The app looks for Petra in the wallet-adapter list and otherwise falls back to a general browser Aptos provider. Connecting a wallet does not start a game automatically.

The user must approve the start-game transaction and each guess in the Petra extension. The contract records both operations under the connected wallet address. Statistics are loaded for that address when it reconnects. Disconnecting the wallet stops live transactions until another account is connected.

6.2 Burner Mode

A burner is a browser-managed Aptos development account. It is designed for convenient testing rather than long-term key custody.

On the first initialization, `getOrCreateBurnerWallet()` generates an Aptos account and Ed25519 private key, then stores the key in browser local storage under `wordle_burner_key`. On later page loads, the app reconstructs the same account from that key. The application also attempts to fund the account through the Aptos devnet faucet.

When the user starts a game or submits a guess, the frontend builds an Aptos transaction, signs it locally with the reconstructed burner account, and sends it through the SDK. No Petra popup appears. After confirmation, the frontend reloads the board and statistics from contract views.

The local private key is persistent but recoverability is limited. Clearing browser storage creates a new burner identity, and clearing storage may therefore make the previous on-chain account and its local key inaccessible. A burner private key should not be used for real assets, shared, or treated as a secure wallet backup.

Both Petra and burner mode call the same contract entry functions:

```
<contract-address>::wordle::start_game
<contract-address>::wordle::submit_guess
```

Faucet funding supplies gas on supported networks. If faucet funding is unavailable or exhausted, the burner may have insufficient APT and live transactions will fail until it is funded through another faucet or funded account.

6.3 Simulation Mode

Simulation mode is independent of Aptos and requires no wallet or gas. Its answer is fixed to `SMART`, and feedback uses the contract's two-pass letter algorithm. Simulation statistics are stored under `wordle:simulation-stats` in browser local storage. They remain separate from Petra and burner statistics and are lost only when browser storage for this site is cleared.

Mode changes reset current board state but do not merge or overwrite statistics belonging to another account or mode. Petra and burner records remain on-chain; simulation records remain local.

6.4 Board and Keyboard

The board has six rows of five tiles. Gray means absent, yellow means misplaced, and green means exact. The keyboard retains the strongest feedback seen for every letter, with gray $<$ yellow $<$ green. After failure, the frontend requests and displays the target word.

7 Statistics

For G games played and W games won, the win percentage is

$$P = \begin{cases} 100\frac{W}{G}, & G > 0, \\ 0, & G = 0. \end{cases}$$

The six-entry distribution counts wins on attempts one through six. Statistics are stored separately for each Aptos account; browser mode changes do not overwrite another account's on-chain statistics.

8 Deployment and Validation

The Move package is compiled and tested with:

```
cd contracts
aptos move compile
aptos move test
```

Publish and initialize using:

```
./deploy.sh --initialize
APTOS_PROFILE=wordle-devnet ./deploy.sh --initialize
./deploy.sh --new-profile wordle-devnet-v2 --initialize
```

The script publishes under the selected account, initializes Wordle, writes the module address and network to `frontend/.env.local`, and may remove an old local profile. Removing a profile deletes local credentials; it does not delete the on-chain account or module.

Move resources or function signatures cannot be changed incompatibly in an upgraded module. Such changes require a fresh account and deployment, after which the frontend environment must point to the new address. Deterministic Move tests should cover feedback and duplicate letters; randomness should not be tested by assuming a particular random index.